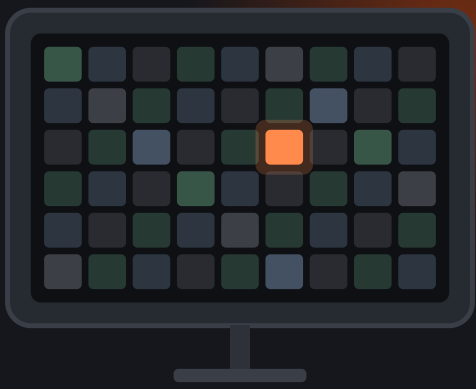
MANUAL DE CAMPO · LOTE 1 · PRÉVIA



Estado & Tela

Aplicações web reativas com React, do DOM manipulado à mão até uma aplicação completa, autenticada e testada, construída com agentes de IA sem perder o controle do código.

ORGANIZADO POR

Prof. Rogério Silva e Claude Opus 5.5

2026 · TEC.1052 · ADS · IFPI

React 19.3 · Vite 8 · TypeScript
React Router 8 · TanStack Query 5 · Vitest 5 · Playwright

Sumário

0 **Introdução** P. 3

PARTE I · A MUDANÇA DE PARADIGMA

1 **Do DOM à mão às interfaces reativas** P. 7

2 **O primeiro projeto em vinte minutos** P. 18

3 **Como o React funciona por dentro** P. 28

PARTE II · OS FUNDAMENTOS

4 Componentes, JSX e composição LOTE 2

5 Props, eventos e fluxo de dados LOTE 2

6 Estado, formulários e interações LOTE 3

7 Efeitos: sincronizar com o mundo lá fora LOTE 3

PARTE III · ARQUITETURA

8 Estrutura de projeto por funcionalidade LOTE 4

9 Estilo, tokens e acessibilidade LOTE 4

10 Componentes reutilizáveis e design system LOTE 4

11 Prop drilling e Context LOTE 5

12 Reducer e transições de estado LOTE 5

PARTE IV · A APLICAÇÃO COMPLETA

13 Rotas e navegação LOTE 6

14 Backend, HTTP e estado remoto LOTE 6

15 Autenticação, sessão e autorização na interface LOTE 7

16 Testes, depuração e desempenho LOTE 7

PARTE V · RENDERIZAÇÃO MODERNA

17 Servidor, cliente e a fronteira entre eles LOTE 8

18 Next.js: visão geral e próximo livro LOTE 8

PARTE VI · ENTREGA E REFERÊNCIA

19 A entrega com agentes, do plano ao PR LOTE 9

20 Guia de consulta, régua e glossário LOTE 9

ANTES DE COMEÇAR

00 Introdução

Para quem é este livro, a ideia que ele defende e como cada capítulo funciona.

Você já sabe fazer uma página reagir a um clique. Este livro trata do passo seguinte: fazer uma aplicação inteira continuar certa quando ela tiver trinta telas, várias pessoas no código e um agente de IA escrevendo boa parte dele.

É para quem já constrói páginas com HTML, CSS e JavaScript, manipula o DOM, consome uma API com `fetch` e já escreveu um backend REST com Express e TypeScript. Nada disso é ensinado de novo. Se você cursa TEC.1052, guarde por perto o front-end do Mini-Prontuário que escreveu em JavaScript puro: ele é o ponto de partida do capítulo 1.

> **A tela é uma função do estado. Quase todo bug de front-end é um estado guardado no lugar errado, duplicado ou sincronizado à mão.**

Por isso o livro não é uma lista de APIs do React. Cada capítulo responde, de um ângulo diferente, a uma pergunta: **onde este estado mora?**



Fig. 0.1 — Os quatro lugares onde um estado pode morar. Hooks, contexto, reducer, rotas e bibliotecas aparecem no livro como respostas a esta pergunta.

O laboratório

> H+ **O livro inteiro acontece num hospital**
>
> Ao lado de cada leito há um monitor. Ninguém redesenha a tela dele à mão: os sensores mudam um número e o monitor atualiza só a faixa que mudou. É o modelo mental do React, e o mundo de todas as analogias do livro.

O projeto é o cliente React do **Mini-Prontuário**, que conversa com a API do ebook *Crachá & Porta*: sessão em cookie, permissões por papel e uma suíte de ataques. A interface é nova; o servidor é real desde o primeiro dia. São três repositórios, `estado-e-tela-api`, `estado-e-tela-web` (uma tag git por capítulo) e `estado-e-tela-ebook`, com este livro, os roteiros de medição e as saídas. Todos os pacientes são fictícios.

Como cada capítulo funciona

A ordem é sempre a mesma: o **problema** num sintoma do Mini-Prontuário; o **modelo mental**, com a caixa *Onde a analogia para*; o mecanismo **por dentro** e **na prática**; o **erro clássico** e a **prova**, um teste que falha sem a correção; o trabalho **com agentes de IA**; e o **fechamento**, com exercícios em três níveis.

Este livro faz parte de uma série. O *Desenvolvimento Dirigido por IA*, o Dev de IA, ensina o método de trabalhar com agentes; aqui ele é aplicado ao React,

sem repetição. Quando um procedimento está detalhado lá, uma caixa **Ponte · Dev de IA** resume e aponta o capítulo. Os prompts seguem os formatos da disciplina:

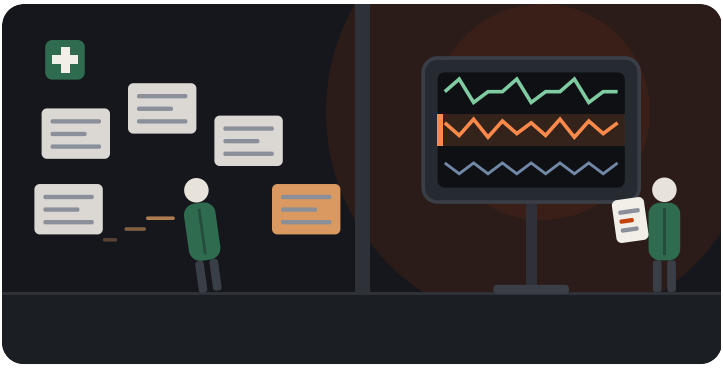
FORMATO	QUANDO	PARTES
Agente	a IA edita o projeto	Goal · Context · Constraints · Done when
C-P-T-R-F-A	conversa no chat	Contexto · Papel · Tarefa · Restrições · Formato · Aceitação
V.E.R.M.	antes de aceitar a resposta	Verifica · Explica · Reduz · Marca

O livro traz o prompt de agente; as variantes de cada capítulo ficam em `prompts/capNN.md`, no repositório da tela.

A régua e as convenções

O capítulo 20 lista **36 competências** de um desenvolvedor sênior de front-end, cada uma com a evidência exigida. Marcar 27 com evidência é chegar a **75%**. Cada capítulo diz quais itens entrega.

- **Dado** é fato externo, com fonte. **Medido** é saída real de código que rodamos, guardada em `saidas/`; nenhuma foi escrita à mão.
- Todo código impresso sai do git, na tag indicada na barra do bloco, em linhas de até 46 caracteres.



PARTE I · A MUDANÇA DE PARADIGMA

01

Do DOM à mão às interfaces reativas

O trabalho de manter a tela em dia com os dados não some quando você adota React. Ele muda de dono.

Terça, 14h. A enfermeira dá alta para Maria Souza, paciente de alto risco da Ala B. A linha some, o total cai de 6 para 5, e o painel continua dizendo: *alto risco: 2*.

Ninguém escreveu um erro; escreveram uma atualização a menos. A tela tinha três lugares que dependiam da mesma lista, e o código da alta lembrou de dois.

1.1

Como chegamos aqui

Em 2005, o AJAX deixou a página pedir dados sem recarregar. Em 2006, o jQuery tornou a manipulação do DOM o jeito padrão de programar a tela. Em 2010, Backbone e AngularJS popularizaram as aplicações de página única. Em 2013 veio o React, com uma ideia diferente: a pergunta deixou de ser *como alterar a tela* quando um dado muda e passou a ser *como a tela deve ficar* para um estado qualquer. Vue e Svelte seguiram a mesma ideia com outra mecânica, e em 2026 o React passou a ser mantido pela React Foundation, na Linux Foundation.

1.2

A Ala B em três versões

O laboratório está em `laboratorio/cap01`, na tag `cap01`: três páginas com a mesma tela, sem build. Seis pacientes fictícios, uma busca, um campo de anotação e um botão de alta por paciente, e um resumo com o total e quantos são de alto risco. Sirva a pasta com `npx serve laboratorio/cap01` e abra cada página.

Cada uma traz no topo um **medidor**, um `MutationObserver` que conta os elementos que entram e saem do DOM e os textos ou atributos trocados. É ele que transforma "o React é eficiente" em número.

1.3

Versão 1: você diz como mudar

A primeira versão é **imperativa**: o código dá ordens ao DOM, passo a passo. Na montagem ele preenche o total e o alto risco; na alta, remove a linha e corrige o total:

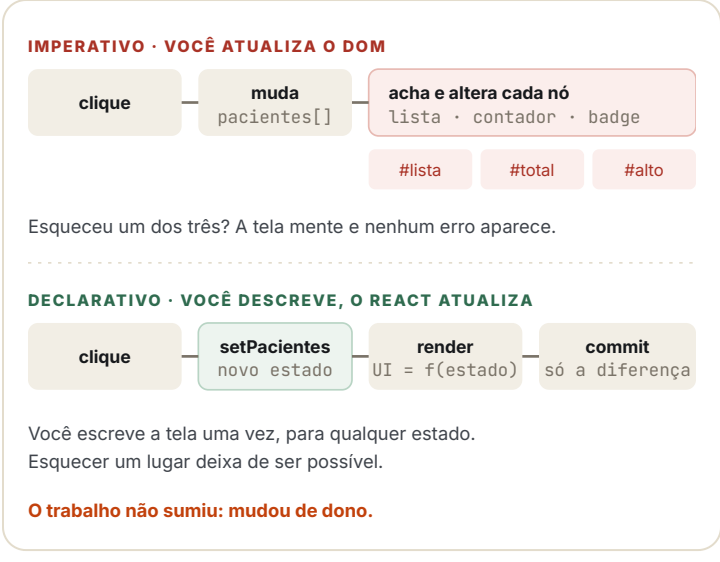


Fig. 1.1 — Quem atualiza a tela. No imperativo, cada evento precisa encontrar e alterar cada nó afetado. No declarativo, você descreve a tela para qualquer estado, e o React calcula o que mudar.

1-imperativo.html · alta cap01

```
// alta: tira a linha e corrige o total
lista.addEventListener("click", (e) => {
  if (e.target.tagName !== "BUTTON") return;
  const li = e.target.closest("li");
  pacientes = pacientes
    .filter((p) => p.id !== li.dataset.id);
  li.remove();
  total.textContent = pacientes.length;
});
```

O código precisa saber quais nós existem, quais dependem de `pacientes` e em que ordem atualizá-los. Com três dependências já houve esquecimento; uma tela real tem dezenas.

1.4

Versão 2: estado e `render()`

A segunda versão é a que a disciplina ensinou: um objeto `estado` e uma função que redesenha a tela inteira a partir dele.

2-render.html · render() cap01

```
// a tela inteira sai do estado, sempre
function render() {
  const t = estado.busca.toLowerCase();
  const vis = estado.pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
  const alto = estado.pacientes
    .filter((p) => p.risco === "alto").length;
  app.innerHTML = `
    <p>${estado.pacientes.length} pacientes
```

```
      · alto risco: ${alto}</p>
    <ul>${vis.map((p) => `
      <li data-id="${p.id}">
        <b>${p.nome}</b> ${p.leito}
        <input placeholder="anotação">
        <button>Alta</button>
      </li>`).join("")}
    </ul>`;
}
```

Ela já pensa do jeito certo: o alto risco é **calculado** a cada render e nunca fica errado. O problema é o `innerHTML`, que joga fora todos os nós e cria outros. Cada tecla na busca reconstrói a lista, e o que estava dentro dos nós antigos vai junto: o texto das anotações, o foco.

1.5

Versão 3: React

A terceira mantém a ideia da segunda e troca o mecanismo. O React vem do `esm.sh` por um `importmap`:

3-react.html · Ala — cap01

```
// a tela é uma função do estado
function Ala() {
  const [busca, setBusca] = useState("");
  const [pacientes, setPacientes] =
    useState(PACIENTES);

  const t = busca.toLowerCase();
  const vis = pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
```

```
  const alto = pacientes
    .filter((p) => p.risco === "alto").length;
  const alta = (id) => setPacientes(
    pacientes.filter((p) => p.id !== id));

  return h("div", null,
    h("input", {
      placeholder: "buscar", value: busca,
      onChange: (e) =>
        setBusca(e.target.value),
    }),
    h("p", null,
      `${pacientes.length} pacientes`
      + ` · alto risco: ${alto}`),
    h("ul", null, vis.map((p) =>
      h("li", { key: p.id },
        h("b", null, p.nome), ` ${p.leito} `,
        h("input",
          { placeholder: "anotação" }),
        h("button",
          { onClick: () => alta(p.id) },
          "Alta")))));
}
```

- `useState` guarda um valor entre renders e devolve a função que o troca. `setPacientes` não mexe no DOM: avisa o React de que o estado mudou.
- `h` é o `createElement` do React. Ele devolve uma *descrição* do que deveria existir, não um nó. O JSX vira exatamente estas chamadas (capítulo 3).
- `key: p.id` diz qual paciente cada linha representa.

A função `Ala` não sabe se roda pela primeira ou pela centésima vez: recebe o estado e devolve a tela. Quem

compara a descrição nova com a anterior e aplica a diferença é o React.

1.6

O que o medidor mostra

As três páginas passaram pelo mesmo roteiro num Chromium: anotar "ver febre" em João Lima, digitar `o` na busca, limpar a busca e dar alta a Maria Souza.

```
medicoes/cap01.py · Medido

Chromium 141.0.7390.37

versão     busca o     alta Maria  nota    alto
imperativo +0 -0 ~2    +0 -4 ~1    fica    2
render()   +18 -26 ~0  +22 -26 ~0  some    1
React      +0 -8 ~1    +0 -4 ~1    fica    1

+ elementos que entram  - que saem
~ textos ou atributos trocados
```

- **Busca.** O imperativo esconde duas linhas (dois atributos `hidden`). O `render()` destrói 26 elementos e cria 18. O React remove só as duas linhas que saíram e troca um atributo, o `value` da busca.
- **Alta.** Imperativo e React fazem o mesmo trabalho no DOM, e o imperativo deixa o alto risco em 2.
- **Nota.** Só o `render()` apagou a anotação.

> **O imperativo economiza e erra. O render() acerta e destrói. O React acerta e economiza.**

Eficiente não quer dizer mais rápido: na alta, o React ainda executou `Ala` e comparou duas descrições da tela. O ganho é deixar de escrever, e de esquecer, cada atualização à mão. O capítulo 16 mede quando o tempo passa a importar.

1.7

Declarativo, estado e reatividade

Declarativo é descrever o resultado para um estado e deixar o mecanismo decidir os passos. **Estado da interface** são os dados que, se mudarem, mudam o que aparece. **Reatividade** é a tela acompanhar o estado sem você escrever a atualização. No React ela é explícita: a tela só muda quando você chama o `set` do estado e o componente roda de novo. Vue, Svelte e Solid rastreiam sozinhos o que cada trecho da tela lê; são escolhas diferentes para o mesmo objetivo.

H+ **O monitor de beira-leito**

A enfermeira não redesenha a tela do monitor. Os sinais vitais mudam e o monitor atualiza a faixa que mudou. Os sinais vitais são o estado; a tela do monitor é a interface.

ONDE A ANALOGIA PARA

O monitor real lê os sensores o tempo todo. O React não observa nada: só calcula a tela de novo quando alguém chama o `set` de um estado. Mudar uma variável comum não muda a tela.

1.8

Biblioteca, não framework

O React transforma estado em interface. Rotas, busca de dados, build de produção e renderização no servidor vêm de outras ferramentas ou de um **framework** construído sobre ele.

DADO · REACT.DEV, FEV. 2025

O Create React App foi descontinuado. A documentação recomenda frameworks (Next.js, React Router, Expo) para produção e ferramentas como o Vite para montar a aplicação do zero e aprender o básico.

Este livro escolhe o caminho do zero com Vite de propósito: cada peça entra quando o problema dela aparece, e você vê o que um framework faz antes de delegar. O Next.js aparece nos capítulos 17 e 18 e terá um ebook próprio.

1.9

Com agentes de IA

ARMADILHA DE IA

Dois vícios imperativos aparecem em código gerado: `document.querySelector` dentro de componente para "atualizar um contador", que briga com o React pelo mesmo DOM; e HTML montado em string com `dangerouslySetInnerHTML`, que reabre a porta para XSS que o React fecha ao escapar todo texto.

PROMPT · AGENTE · CAP 1

```
Goal
Reescrever a tela de pacientes do meu
front-end vanilla como função do estado,
com React por importmap, sem build.

Context
@laboratorio/cap01/3-react.html (modelo)
@<sua tela vanilla>.html

Constraints
- sem document.querySelector em componente
- sem innerHTML e sem
  dangerouslySetInnerHTML
- contagens calculadas no render
- key de lista é o id do registro
- fora de escopo: estilo, API, build

Done when
- anotar e buscar: a anotação fica
- após a alta, todas as contagens batem
```

```
- no medidor, a alta só remove a linha
  do paciente e troca textos
```

Exercícios

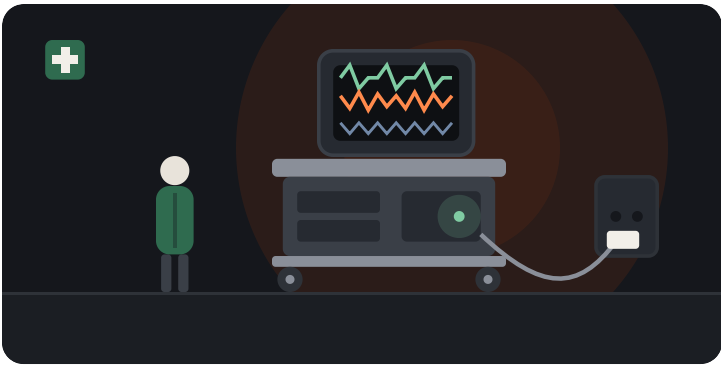
ESSENCIAL Corrija a alta na versão imperativa. Depois liste o que precisaria mudar se a tela ganhasse um terceiro número, como pacientes em isolamento.

INTERMEDIÁRIO Na versão `render()`, mova a busca para dentro do `#app` e explique, com o medidor, por que o campo perde o foco a cada tecla.

DESAFIO Na versão React, troque `key: p.id` por `key: i`. Anote em João Lima, dê alta a Maria Souza e descreva o que aconteceu. O capítulo 3 explica.

Régua: 01. **Tag:** `cap01`. **Prompts:** revisão e C-P-T-R-F-A em `prompts/cap01.md`.

Fontes. react.dev: *Sunsetting Create React App* (fev. 2025), *Creating a React App*, *Introducing the React Foundation* (out. 2025; fundação lançada em fev. 2026). MDN: *MutationObserver*. Medição em `saidas/c1-ala.txt`, Chromium 141, com React 19.3.1 servido localmente nas URLs do importmap.



PARTE I · A MUDANÇA DE PARADIGMA

02

O primeiro projeto em vinte minutos

Do terminal vazio a um projeto versionado, ligado à API e pronto para um agente trabalhar nele.

A página do capítulo 1 funcionou sem build, mas não escala: sem JSX, sem TypeScript, sem pacotes, sem build de produção. Este capítulo monta o projeto de verdade.

Você precisa do **Node 24** (ou 22.22+), Git, um editor e a extensão **React Developer Tools**. O Vite 8 aceita Node 20.19+, mas o React Router 8, que entra no capítulo 13, exige 22.22+; começar no 24 evita trocar de versão no meio do livro.

2.1

Gerando o projeto

terminal

```
npm create vite@latest mini-prontuario-web \
  -- --template react-ts
cd mini-prontuario-web && npm install
npm run dev     # http://localhost:5173
```

O `index.html` é a única página: um `<div id="root">` e o script de `src/main.tsx`, que cria a raiz do React. O `vite.config.ts` configura o Vite, os `tsconfig*.json` os tipos, e o `.oxlintrc.json` o linter, já com as regras dos Hooks.

DADO · O TEMPLATE DE OUTUBRO DE 2026

Com `create-vite` 9.3.1, o template usa **oxlint** em vez de ESLint e fixa o **TypeScript 6.0**: o 7.0, com compilador nativo, ainda não tem a API que parte das ferramentas usa. Não há `"strict": true` porque o modo estrito virou o padrão:

terminal · Medido

```
$ npx tsc -p tsconfig.app.json --noEmit
src/_t.ts(1,19): error TS7006: Parameter 'x'
  implicitly has an 'any' type.
```

2.2

Scripts e o portão

`dev` serve a tela e a atualiza quando você salva; `build` confere os tipos e gera `dist/`; `preview` serve o build; `lint` roda o oxlint. O projeto acrescenta `gate`, que roda tudo o que precisa passar antes de qualquer entrega:

```
terminal · Medido

$ npm run gate
> tsc -b && oxlint && vite build

vite v8.3.5 building client environment for
production...
✓ 16 modules transformed.
…
index-DeCkmCJv.js   219.85 kB | gzip: 68.76 kB
✓ built in 108ms
```

O `tsc -b` não gera arquivos (o `tsconfig` tem `noEmit`): quem transforma TypeScript em JavaScript é o Vite, e ele não confere tipos. Por isso o `dev` mostra a tela mesmo com erro de tipo, e o `gate` não deixa esse erro passar.

2.3

O ponto de entrada

src/main.tsx cap02

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!)
  .render(
    <StrictMode>
      <App />
    </StrictMode>,
  );
```

`createRoot` liga o React a um nó do DOM, e `render` faz o primeiro desenho. `StrictMode` só age em desenvolvimento: chama cada componente duas vezes por render e monta e desmonta cada efeito uma vez a mais, para revelar código que depende de rodar uma vez só. Os capítulos 3 e 7 mostram o que ele pega.

2.4

A tela e a API na mesma origem

A tela roda em `:5173` e a API em `:3000`. Portas diferentes são **origens** diferentes para o navegador, o que traria CORS e regras de cookie entre sites. O **proxy** do Vite evita isso:

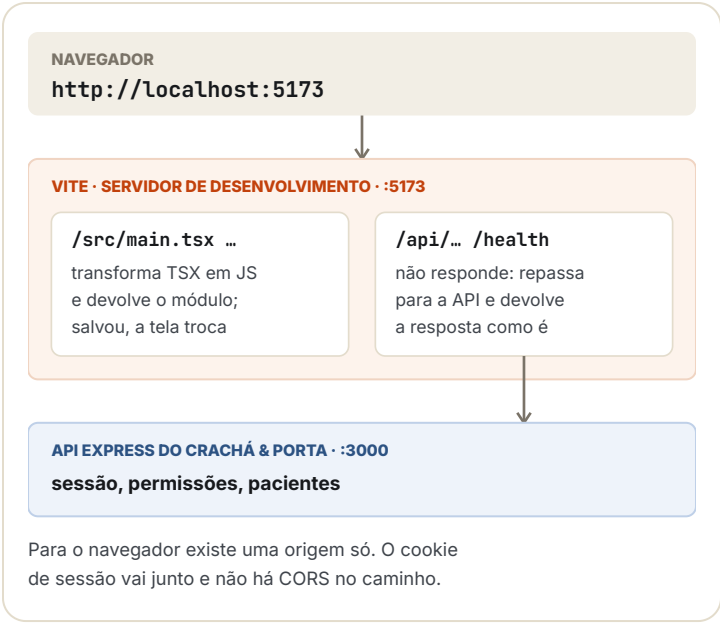


Fig. 2.1 — O navegador só conhece `localhost:5173`. O Vite serve o código da tela e repassa para a API o que começa com `/api` ou `/health`.

vite.config.ts cap02

```ts
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";

// O Vite serve a tela em :5173 e repassa para
// a API em :3000 tudo que começa com /api.
// Para o navegador, é tudo uma origem só.
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": "http://localhost:3000",
      "/health": "http://localhost:3000",
    },
```

```
  },
});
```

Em produção não há Vite: o build vira arquivos estáticos e o servidor que os publica faz o papel do proxy. O código não muda, porque nunca usa endereço de servidor:

REGRA · UI-00

O front chama a API por caminho relativo (`/api/...`). Verificação: `grep -rn "localhost:3000" src` não encontra nada.

O `App` do template virou um verificador: um botão cujo **handler de evento** chama `fetch("/health")` e guarda o resultado em estado. Buscar dados ao abrir a tela pede `useEffect`, e há bons motivos para esperar até o capítulo 7. A sessão também passa pelo proxy: este roteiro entra como Ana pela tela e consulta quem está logado.

medicoes/cap02_cookie.py · Medido

```
chromium 141.0.7390.37 · http://localhost:5173/
tela: API ok (via proxy /health)
POST /api/auth/login  -> 200
cookie __Host-sessao: httpOnly=True secure=True
sameSite=Lax
document.cookie       -> ''
GET  /api/auth/me     -> 200 ana@hospital.dev
```

O cookie chegou `HttpOnly`, `Secure` e `SameSite=Lax`, o JavaScript da página não o lê, e mesmo assim o pedido seguinte saiu autenticado: o navegador anexa o cookie sozinho. O capítulo 15 constrói a sessão da tela sobre isso; o porquê de cada atributo está no *Crachá & Porta*, capítulo 5.

Para inspecionar, a extensão React Developer Tools acrescenta as abas **Components** (árvore, props e estado) e **Profiler** (quem renderizou e quanto levou). O Vite entrega *source maps*: os *breakpoints* param no `.tsx` original.

2.5

Git e o contexto do agente

O primeiro commit vem antes da primeira feature, com quatro arquivos que o agente lê antes de qualquer tarefa:

ARQUIVO	O QUE GUARDA
`AGENTS.md`	Comandos, como o projeto funciona e regras; formato que Codex e agy leem
`CLAUDE.md`	Uma linha, `@AGENTS.md`: o Claude Code importa o mesmo conteúdo
`INVARIANTES.md`	Regras com verificação; começa com a UI-00
`IA.md`	Cada tarefa feita com agente: pedido, verificação, o que foi recusado

PONTE · DEV DE IA, CAPS. 3 A 5

Por que o contexto deve ser curto e como um invariante ganha verificação executável: o mapa dos cinco arquivos (cap. 3), `CLAUDE.md` e `AGENTS.md` (cap. 4) e `INVARIANTES.md` (cap. 5).

2.6

Com agentes de IA

A primeira tarefa num projeto novo é conferir se o agente entendeu o projeto, em modo de plano, sem editar nada. O conteúdo do prompt é o mesmo nas três ferramentas; muda como você planeja, revisa e roda sem interação:

PASSO	CLAUDE CODE	CODEX · AGY
Planejar	Shift+Tab (Plan Mode)	`/plan` · modo de plano
Revisar	`/code-review`	`/review` · subagente
Sem interação	`claude -p`	`codex exec` · `agy -p`

PROMPT · AGENTE (MODO DE PLANO) · CAP 2

```
Goal
Explicar o caminho de um clique em
"Verificar API" até a resposta da API.

Context
```

```
@AGENTS.md @vite.config.ts @src/
A API roda em :3000 (outro repositório).

Constraints
- só leitura: não edite nada
- cite arquivo e linha de cada passo
- diga o que muda em produção

Done when
- cita main.tsx, App.tsx, vite.config.ts
- explica por que não há CORS no caminho
- git status continua limpo
```

ARMADILHA DE IA

Diante de um erro de CORS, o agente costuma propor `fetch("http://localhost:3000/...")` e liberar `origin: "*"` na API. As duas mudanças funcionam e estão erradas: a primeira quebra a UI-00 e a produção, a segunda abre a API para qualquer site. O `AGENTS.md` já proíbe mexer na API e no Vite. Comandos de agente mudam rápido; confira na versão instalada.

Exercícios

ESSENCIAL Gere o projeto, configure o proxy e faça o botão mostrar **ok**. Pare a API e clique de novo: o que aparece, e por quê?

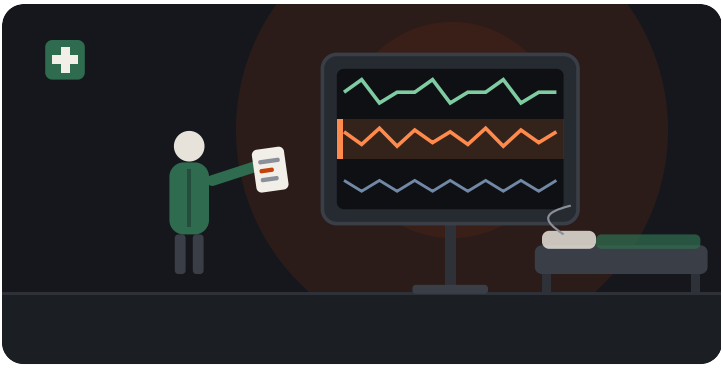
INTERMEDIÁRIO Introduza um erro de tipo no `App.tsx`. Rode `npm run dev` e depois `npm run gate` e explique a diferença.

DESAFIO

Rode `npm run build` e `npm run preview`: o botão funciona. Descubra na documentação do Vite por quê, e escreva a configuração que um servidor de produção (nginx, por exemplo) precisaria para a UI-00 continuar valendo.

Régua: prepara 34 e 35. **Invariantes:** UI-00. **Tag:** `cap02`. **Prompts:** `prompts/cap02.md`.

Fontes. Vite 8 (mar. 2026), *server.proxy*, *preview.proxy*; react.dev: *createRoot*, *StrictMode*; React Router v8; TypeScript 7 (InfoQ). Template de 21 out. 2026. Saídas em `saidas/c2-*.txt`.



PARTE I · A MUDANÇA DE PARADIGMA

03

Como o React funciona por dentro

Gatilho, render e commit, identidade e keys, provados com o motor ligado.

Quarta, 10h. A enfermeira anota "ver febre" no campo de João Lima e dá alta a Maria Souza. A anotação aparece na linha de Ana Clara Reis; João fica com o campo vazio.

O código da alta está certo. O problema está numa linha que parece irrelevante, e para achá-lo é preciso saber o que o React faz entre a sua função devolver a tela e o navegador mostrá-la.

3.1

O projeto do capítulo

Na tag `cap03`, a Ala B virou componentes com JSX e TypeScript, em `src/features/pacientes`. O cartão de cada paciente:

src/features/pacientes/CartaoPaciente.tsx `cap03`

```tsx
import type { Paciente } from "./tipos.ts";

interface Props {
  paciente: Paciente;
  onAlta: (id: string) => void;
}

export function CartaoPaciente(
  { paciente, onAlta }: Props,
) {
  const { id, nome, leito } = paciente;
  return (
    <li>
      <b>{nome}</b> {leito}
      <input
        aria-label={`Anotação de ${nome}`}
        placeholder="anotação"
      />
      <button onClick={() => onAlta(id)}>
        Alta
      </button>
    </li>
  );
}
```

A `Ala` guarda `busca` e `pacientes` em estado e calcula o resto, exatamente como a versão 3 do capítulo

1, agora em JSX.

3.2

JSX é açúcar

O navegador não entende JSX. Antes de chegar a ele, cada tag vira uma chamada de função, como o `h` do capítulo 1. É isto que o Vite entrega quando o navegador pede o cartão:

servidor do Vite · Medido

```
GET /src/features/pacientes/CartaoPaciente.tsx
(trecho; metadados de depuração viraram …)

export function CartaoPaciente({ paciente, onAlta
}) {
  const { id, nome, leito } = paciente;
  return _jsxDEV("li", { children: [
    _jsxDEV("b", { children: nome }, …),
    " ",
    leito,
    _jsxDEV("input", {
      "aria-label": `Anotação de ${nome}`,
      placeholder: "anotação"
    }, …),
    _jsxDEV("button", {
      onClick: () => onAlta(id),
      children: "Alta"
    }, …)
  ] }, …);
}
```

Em desenvolvimento a função se chama `jsxDEV` e carrega arquivo e linha de cada tag para as mensagens

de erro; em produção, `jsx`. As chaves do JSX viram expressões comuns, os filhos viram a prop `children`, e o resultado é um **elemento**: um objeto que *descreve* um `<li>`. Nenhum nó do DOM foi criado.

3.3

Gatilho, render, commit



Fig. 3.1 — As etapas de toda atualização. O **gatilho** é a montagem inicial ou um `set`. No **render**, o React chama os componentes e recebe a árvore nova. No **commit**, compara com a anterior e aplica no DOM só o que mudou. A pintura já é do navegador; os efeitos são o capítulo 7.

H+ **A passagem de plantão**

Na troca de turno, a enfermeira compara o quadro de ontem com a situação de hoje e anota só o que mudou. Olhar a situação é o render; anotar a diferença é o commit.

ONDE A ANALOGIA PARA

O plantão tem hora marcada; o render só acontece com gatilho e pode terminar sem diferença nenhuma. O React pode até descartar um render pela metade para atender algo mais urgente. O commit, nunca: é aplicado inteiro, de uma vez.

3.4

Renderizar não é mexer no DOM

Para separar as duas coisas, o projeto tem o `Renderometro`, construído sobre o `<Profiler>` do React: a cada commit em que algo dentro dele renderizou, o React chama `onRender` com o nome, a fase e o tempo.

src/lab/Renderometro.tsx cap03

```
export function Renderometro(
  { id, children }: Props,
) {
  return (
    <Profiler
      id={id}
      onRender={(quem, fase, ms) => {
```

```
        const linha = `${quem} · ${fase}`
          + ` · ${ms.toFixed(2)} ms`;
        registros.push(linha);
        console.log(`[render] ${linha}`);
      }}
    >
      {children}
    </Profiler>
  );
}
```

A lista e cada cartão estão dentro de um `Renderometro`. Um roteiro no Chromium cruza o que ele anotou com o que um `MutationObserver` viu no DOM:

medicoes/cap03.py · Medido

```
digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  Lista           update
  DOM: +0 -8 ~1
…
anotar em João Lima
  DOM: +0 -0 ~0
```

Uma tecla na busca fez a lista e os quatro cartões visíveis renderizarem de novo. No DOM, porém, só saíram as duas linhas que deixaram de bater, oito elementos, e mudou o `value` da busca. Os cartões foram *calculados* de novo, deram o mesmo resultado, e o commit não os tocou.

Digitar na anotação de João não provocou render nenhum: aquele campo não tem estado no React, o texto vive no `<input>` do DOM. Guarde isso para a seção 3.6.

3.5

O "Virtual DOM", com precisão

"Virtual DOM" é o apelido da árvore de elementos que cada render devolve. Ele **não é uma cópia do DOM mantida em sincronia**: é a descrição produzida por um render, comparada com a do render anterior. E **não é sempre mais rápido** que mexer no DOM à mão: no capítulo 1, imperativo e React fizeram o mesmo trabalho no DOM, e o React ainda executou componentes. O que ele entrega é correção sem esforço manual.

A comparação, a **reconciliação**, segue regras simples:

NA MESMA POSIÇÃO	O QUE O REACT FAZ
Mesmo tipo	Mantém o nó e o estado; atualiza o que mudou nas props
Tipo diferente	Desmonta o que havia, com o estado, e monta do zero
Em listas, com `key`	Casa os itens pela key, não pela posição

3.6

Identidade: o estado mora na posição

A regra da documentação é curta: **o React mantém o estado de um componente enquanto ele for renderizado na mesma posição da árvore**. Ser "o mesmo", para o React, é estar no mesmo lugar, com o mesmo tipo e, em listas, a mesma key. Na tag `cap03-antes`, a lista usava o índice como key:

```
src/features/pacientes/ListaPacientes.tsx                cap03-antes → cap03

…
      <ul>
-       {pacientes.map((p, i) => (
-         <Renderometro key={i} id={p.nome}>
+       {pacientes.map((p) => (
+         <Renderometro key={p.id} id={p.nome}>
            <CartaoPaciente
```



Fig. 3.2 — Com key por índice, o React vê que a última posição sumiu e que as outras só trocaram de props. O `<input>` da posição 1 fica no DOM, com o texto, mostrando agora Ana Clara.

O texto nunca esteve no React; estava no nó. Com key por índice, os nós são reaproveitados pela posição e o texto fica preso a eles. Com o `id`, o React sabe que `p1` saiu e que `p2` continua `p2`. O teste descreve o que a enfermeira faz, sem olhar para dentro do componente:

src/features/pacientes/Ala.test.tsx cap03

```
test("anotação acompanha o paciente",
  async () => {
    render(<Ala />);
    await userEvent.type(
      nota("João Lima"), "ver febre");
    await darAlta("Maria Souza");
    expect(nota("João Lima").value)
      .toBe("ver febre");
  });
```

CAP03-ANTES · KEY={I}

```
× anotação acompanha o paciente
→ expected '' to be 'ver febre'
```

CAP03 · KEY={P.ID}

```
✓ anotação acompanha o paciente
✓ alto risco cai na alta
```

É a **prova de revert** da série: o teste falha sem a correção e passa com ela. As saídas completas estão em `saidas/c3-key-*.txt`. Trocar a key de propósito também serve para zerar um componente; o capítulo 6 usa isso.

REGRA · UI-01

Lista usa a key do domínio (o `id`), nunca o índice. Verificação: o teste "anotação acompanha o paciente".

3.7

Montar, atualizar, desmontar

Montar é aparecer numa posição: nós criados, estado no valor inicial. **Atualizar** é renderizar de novo no mesmo lugar. **Desmontar** é sair da árvore, levando nós e estado. Na medição completa, Ana Clara e Luzia voltam como `mount` quando a busca é limpa: tinham sido desmontadas.

Para isso funcionar, o render precisa ser **puro**: as mesmas props e o mesmo estado produzem a mesma tela, e renderizar não altera nada fora do componente. O `StrictMode` testa a promessa chamando cada componente duas vezes em desenvolvimento, e é por isso que o `Renderometro` conta commits pelo `Profiler` em vez de somar uma variável no corpo do componente. Internamente, o render é dividido em unidades de trabalho, as *fibers*, o que permite pausá-lo; recursos como `startTransition` usam isso e aparecem quando forem úteis.

3.8

Com agentes de IA

ARMADILHA DE IA

Agentes geram `key={index}` com frequência, porque é o exemplo mais curto, e às vezes `key={Math.random()}`, que é pior: cada render remonta tudo e apaga todo estado da lista. Nenhum dos dois quebra o build. A UI-01 e o teste pegam ambos.

Para um bug como o da abertura, o primeiro prompt pede diagnóstico com evidência, não correção:

PROMPT · AGENTE · CAP 3 · DIAGNÓSTICO

```
Goal
Descobrir por que a anotação de um
paciente aparece em outro depois de
uma alta, e provar a causa.

Context
@src/features/pacientes/ @INVARIANTES.md
Passos: anotar em João Lima, dar alta a
Maria Souza; a nota vai para Ana Clara.

Constraints
- antes, um teste que reproduz o defeito
  pelo que o usuário vê, falhando
- depois, a correção mínima
- sem dependência nova

Done when
- npm run gate verde com a correção
- o teste novo FALHA se a correção
```

```
  for revertida
- a causa aponta arquivo e linha
```

> **PONTE · DEV DE IA, CAPS. 9 E 13**
>
> O "falha se revertido" vem do capítulo 9: agentes escrevem com facilidade testes que passam antes e depois da mudança. A revisão em contexto novo, de preferência com outra ferramenta, é o capítulo 13; o prompt dela está em `prompts/cap03.md`.

Exercícios

ESSENCIAL Rode `npm test` em `cap03-antes` e em `cap03`. Explique a diferença em três frases, usando "posição" e "key".

INTERMEDIÁRIO Preveja quais componentes renderizam ao dar alta ao último paciente. Rode com o `Renderometro` e compare. Por que a previsão para key por índice seria diferente?

DESAFIO Troque a key por `Math.random()`, meça uma tecla na busca com `medicoes/cap03.py` e explique o número.

Régua: 02, 03, 04. **Invariantes:** UI-01. **Tags:** `cap03-antes`, `cap03`. **Prompts:** `prompts/cap03.md`.

Fontes. react.dev: *Render and Commit*, *Preserving and Resetting State*, *Rendering Lists*, *Keeping Components Pure*, *<Profiler>*, *<StrictMode>*. Saídas em `saidas/c3-*.txt`.